# FeatureAlphaDropout#

https://pytorch.org/docs/stable/generated/torch.nn.FeatureAlphaDropout.html#

**Description:**

Randomly masks out entire channels. A channel is a feature map, e.g. the jjj-th channel of the iii-th sample in the batch input is a tensor input[i,j]\text{input}[i, j]input[i,j] of the input tensor). Instead of setting activations to zero, as in regular Dropout, the activations are set to the negative saturation value of the SELU activation function. More details can be found in the paper Self-Normalizing Neural Networks . Each element will be masked independently for each sample on every forward call with probability p using samples from a Bernoulli distribution. The elements to be masked are randomized on every forward call, and scaled and shifted to maintain zero mean and unit variance. Usually the input comes from nn.AlphaDropout modules. As described in the paper Efficient Object Localization Using Convolutional Networks , if adjacent pixels within feature maps are strongly correlated (as is normally the case in early convolution layers) then i.i.d. dropout will not regularize the activations and will otherwise just result in an effective learning rate decrease.

## Function Signature

```
class torch.nn.FeatureAlphaDropout(p=0.5, inplace=False)
[source]#

Parameters

Shape:

forward(input)[source]#

Return type
```

**Returns:**

Tensor

## Code Examples

```
>>> m = nn.FeatureAlphaDropout(p=0.2)
>>> input = torch.randn(20, 16, 4, 32, 32)
>>> output = m(input)
```

## Detailed Documentation

### Documentation

Randomly masks out entire channels.

A channel is a feature map, e.g. the jjj-th channel of the iii-th sample in the batch input is a tensor input[i,j]\text{input}[i, j]input[i,j] of the input tensor). Instead of setting activations to zero, as in regular Dropout, the activations are set to the negative saturation value of the SELU activation function. More details can be found in the paper Self-Normalizing Neural Networks .

Each element will be masked independently for each sample on every forward call with probability p using samples from a Bernoulli distribution. The elements to be masked are randomized on every forward call, and scaled and shifted to maintain zero mean and unit variance.

Usually the input comes from nn.AlphaDropout modules.

As described in the paper Efficient Object Localization Using Convolutional Networks , if adjacent pixels within feature maps are strongly correlated (as is normally the case in early convolution layers) then i.i.d. dropout will not regularize the activations and will otherwise just result in an effective learning rate decrease.

In this case, nn.AlphaDropout() will help promote independence between feature maps and should be used instead.

p (float, optional) – probability of an element to be zeroed. Default: 0.5

inplace (bool, optional) – If set to True, will do this operation in-place

Input: (N,C,D,H,W)(N, C, D, H, W)(N,C,D,H,W) or (C,D,H,W)(C, D, H, W)(C,D,H,W).

Output: (N,C,D,H,W)(N, C, D, H, W)(N,C,D,H,W) or (C,D,H,W)(C, D, H, W)(C,D,H,W) (same shape as input).

Runs the forward pass.